

Chapter 4: Descent Methods

MATH60005/70005: Optimisation
Term 1, Academic Year 25-26

Dr Dante Kalise
Department of Mathematics
Imperial College London
dkaliseb@ic.ac.uk

This week we study the gradient descent method, arguably the most relevant algorithm in unconstrained continuous optimization. We discuss its rationale, and how to tune it. We will study its convergence properties, and the importance of the condition number of a matrix to understand the convergence rate of the method.

Descent Directions Methods	1
Taking the Direction of Minus the Gradient	4
Convergence of the Gradient Method	7
The Condition Number	11
Scaled Gradient Method	14
The Gauss-Newton Method	18
The Fermat-Weber Problem and Weiszfeld's Method	21
Newton's Method	22
The Kaczmarz Algorithm	27
Stochastic Gradient Descent	29
Revisiting the Nonlinear Regression Example	31

Descent Directions Methods

Recall that our objective is to find an optimal solution of the problem

$$\min \{f(\mathbf{x}) : \mathbf{x} \in \mathbb{R}^n\} .$$

We have seen that in some particular cases, such as the Linear Least Squares problem from Week 3, it is possible to obtain a direct solution by solving the normal equations.



Unfortunately, this is not the case for an arbitrary nonlinear objective $f(\mathbf{x})$. We will study the solution of this optimization problem by using iterative algorithms of the form

$$\mathbf{x}^{k+1} = \mathbf{x}^k + t^k \mathbf{d}^k, \quad k = 0, 1, \dots,$$

where $\mathbf{d}^k \in \mathbb{R}^n$ is a **direction** and $t^k \in \mathbb{R}$ is called the **stepsize**. We will see that a careful selection of $\mathbf{d}^k$ and t^k will generate a sequence $\{\mathbf{x}^k\}_{k=0}^{\infty}$ converging to a stationary point $\mathbf{x}^*$ such that $\nabla f(\mathbf{x}^*) = 0$ (a very good candidate for solving our original problem). A first important concept is the one of descent direction.

Definition (Descent Direction). *Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be a continuously differentiable function over $\mathbb{R}^n$. A vector $\mathbf{0} \neq \mathbf{d} \in \mathbb{R}^n$ is called a descent direction of f at $\mathbf{x}$ if the directional derivative $f'(\mathbf{x}; \mathbf{d})$ is negative, meaning that*

$$f'(\mathbf{x}; \mathbf{d}) = \nabla f(\mathbf{x})^\top \mathbf{d} < 0.$$



Figure 1: Augustin-Louis Cauchy (1789-1857), French mathematician, physicist and engineer. He made pioneering contributions to several branches of mathematics. In his 1847 paper “Méthode générale pour la résolution des systèmes simultanés” he presented the first formulation of the gradient descent method.

The Descent Property of Descent Directions

Lemma. *Let f be a continuously differentiable function over $\mathbb{R}^n$, and let $\mathbf{x} \in \mathbb{R}^n$. Suppose that $\mathbf{d}$ is a descent direction of f at $\mathbf{x}$. Then there exists $\varepsilon > 0$ such that*

$$f(\mathbf{x} + t\mathbf{d}) < f(\mathbf{x})$$

for any $t \in (0, \varepsilon]$.



Proof. Since $f'(\mathbf{x}; \mathbf{d}) < 0$, it follows from the definition of the directional derivative that

$$\lim_{t \rightarrow 0^+} \frac{f(\mathbf{x} + t\mathbf{d}) - f(\mathbf{x})}{t} = f'(\mathbf{x}; \mathbf{d}) < 0.$$

Therefore, there exists $\varepsilon > 0$ such that

$$\frac{f(\mathbf{x} + t\mathbf{d}) - f(\mathbf{x})}{t} < 0$$

for any $t \in (0, \varepsilon]$, which readily implies the desired result. $\square$

A first version of the descent direction algorithm is presented below.

Algorithm 1: Schematic Descent Direction Method

Initialization: pick $\mathbf{x}^0 \in \mathbb{R}^n$ arbitrarily.

General Step: for any $k = 0, 1, 2, \dots$ execute the following steps:

- 1 Pick a descent direction $\mathbf{d}^k$.
 - 2 Find a stepsize t^k satisfying $f(\mathbf{x}^k + t^k \mathbf{d}^k) < f(\mathbf{x}^k)$.
 - 3 Set $\mathbf{x}^{k+1} = \mathbf{x}^k + t^k \mathbf{d}^k$.
 - 4 If a stopping criteria is satisfied, then STOP and $\mathbf{x}^{k+1}$ is the output.
-

Of course, many details are missing in the above schematic algorithm:

- What is the starting point?
- How to choose the descent direction?
- What stepsize should be taken?
- What is the stopping criteria?

Stepsize Selection Rules

Let's assume for one second that a descent direction has been found. How do we choose the stepsize? There are several options available.

- Constant stepsize: $t^k = \bar{t}$ for any k .
- Exact stepsize: t^k is a minimizer¹ of f along the ray $\mathbf{x}^k + t\mathbf{d}^k$:

$$t^k \in \operatorname{argmin}_{t \geq 0} f(\mathbf{x}^k + t\mathbf{d}^k)$$

¹ We call the operation of selecting the element that minimizes instead of just computing the minimum value the argmin.



- Backtracking (or Armijo rule): the method requires three parameters: $s > 0$, $\alpha \in (0, 1)$, and $\beta \in (0, 1)$. Here we start with an initial stepsize $t^k = s$. While

$$f(\mathbf{x}^k) - f(\mathbf{x}^k + t^k \mathbf{d}^k) < -\alpha t^k \nabla f(\mathbf{x}^k)^\top \mathbf{d}^k$$

set $t^k := \beta t^k$, iterating until achieving the **Sufficient Decrease Property**

$$f(\mathbf{x}^k) - f(\mathbf{x}^k + t^k \mathbf{d}^k) \geq -\alpha t^k \nabla f(\mathbf{x}^k)^\top \mathbf{d}^k .$$

Finding the right t^k is referred in the literature as line search.

Exercise: exact line search for quadratic functions. Find the exact stepsize when $f(\mathbf{x})$ is a quadratic function $f(\mathbf{x}) = \mathbf{x}^\top \mathbf{A} \mathbf{x} + 2\mathbf{b}^\top \mathbf{x} + \mathbf{c}$ where $\mathbf{A}$ is an $n \times n$ positive definite matrix, $\mathbf{b} \in \mathbb{R}^n$ and $\mathbf{c} \in \mathbb{R}$. Let $\mathbf{x} \in \mathbb{R}^n$ and let $\mathbf{d} \in \mathbb{R}^n$ be a descent direction of f at $\mathbf{x}$. The objective is to find a solution to

$$\min_{t \geq 0} f(\mathbf{x} + t\mathbf{d}) .$$

Taking the Direction of Minus the Gradient

In the gradient method we make the choice $\mathbf{d}^k = -\nabla f(\mathbf{x}^k)$. This is a descent direction as long as $\nabla f(\mathbf{x}^k) \neq 0$ since

$$f'(\mathbf{x}^k; -\nabla f(\mathbf{x}^k)) = -\nabla f(\mathbf{x}^k)^\top \nabla f(\mathbf{x}^k) = -\|\nabla f(\mathbf{x}^k)\|^2 < 0$$

In addition to being a descent direction, minus the gradient is also the steepest descent direction.

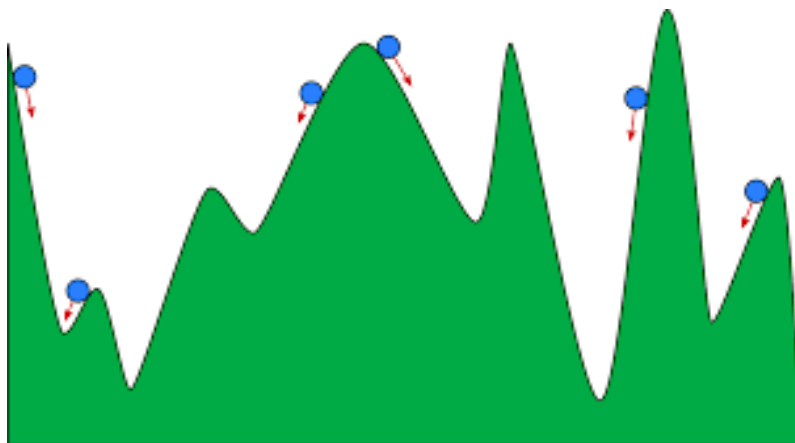


Figure 2: The landscape is $f(\mathbf{x})$ and the balls are ready to move in the direction of $-\nabla f(\mathbf{x})$. What can you say about the points where the balls are expected to end?

Lemma. Let f be a continuously differentiable function and let $\mathbf{x} \in \mathbb{R}^n$ be a non-stationary point ($\nabla f(\mathbf{x}) \neq 0$). Then an optimal solution of

$$\min_{\mathbf{d}} \{f'(\mathbf{x}; \mathbf{d}) : \|\mathbf{d}\| = 1\}$$

is $\mathbf{d} = -\nabla f(\mathbf{x}) / \|\nabla f(\mathbf{x})\|$.

Proof. Since $f'(\mathbf{x}; \mathbf{d}) = \nabla f(\mathbf{x})^\top \mathbf{d}$, we write the problem as

$$\min_{\mathbf{d} \in \mathbb{R}^n} \{\nabla f(\mathbf{x})^\top \mathbf{d} : \|\mathbf{d}\| = 1\}$$

By the Cauchy-Schwarz inequality we have

$$\nabla f(\mathbf{x})^\top \mathbf{d} \geq -\|\nabla f(\mathbf{x})\| \cdot \|\mathbf{d}\| = -\|\nabla f(\mathbf{x})\|.$$

Thus, $-\|\nabla f(\mathbf{x})\|$ is a lower bound on the optimal value of the directional derivative. On the other hand, using the direction $\mathbf{d} = -\frac{\nabla f(\mathbf{x})}{\|\nabla f(\mathbf{x})\|}$ we obtain that

$$f' \left(\mathbf{x}, -\frac{\nabla f(\mathbf{x})}{\|\nabla f(\mathbf{x})\|} \right) = -\nabla f(\mathbf{x})^\top \left(\frac{\nabla f(\mathbf{x})}{\|\nabla f(\mathbf{x})\|} \right) = -\|\nabla f(\mathbf{x})\|$$

and we thus come to the conclusion that the lower bound $-\|\nabla f(\mathbf{x})\|$ is attained at $\mathbf{d} = -\frac{\nabla f(\mathbf{x})}{\|\nabla f(\mathbf{x})\|}$, which readily implies that this is an optimal solution for the descent direction. $\square$

Algorithm 2: The Gradient Method

Initialization: A tolerance parameter $\varepsilon > 0$ and $\mathbf{x}^0 \in \mathbb{R}^n$.

General Step: for any $k = 0, 1, 2, \dots$ execute the following steps:

- 1 Pick a stepsize t^k by a line search procedure on the function

$$g(t) = f(\mathbf{x}^k - t\nabla f(\mathbf{x}^k)).$$

- 2 Set $\mathbf{x}^{k+1} = \mathbf{x}^k - t^k \nabla f(\mathbf{x}^k)$.
 - 3 If $\|\nabla f(\mathbf{x}^{k+1})\| \leq \varepsilon$, then STOP and $\mathbf{x}^{k+1}$ is the output.
-

Example. Solve

$$\min x_1^2 + 2x_2^2$$

using a gradient descent with exact line search. Use the starting guess $\mathbf{x}^0 = (2, 1)$, and a stopping tolerance of $\varepsilon = 10^{-5}$. The iteration converges in 13 steps, and the convergence history is shown in Figure 4. After you try to code it by yourself, you can check the MATLAB code at the end of this document.



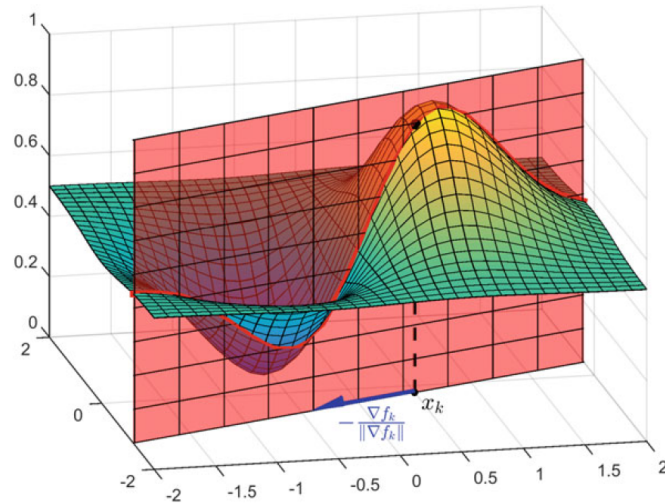


Figure 3: At a given iteration, the current point $\mathbf{x}^k$ and the descent direction $-\frac{\nabla f(\mathbf{x}^k)}{\|\nabla f(\mathbf{x}^k)\|}$ define a plane along which the next iterate $\mathbf{x}^{k+1}$ is sought. The line search procedure defines how far do we move in this direction. A constant stepsize will move a fixed distance along the red line, whereas an exact stepsize will look for the minimizer of the surface constrained to the plane.

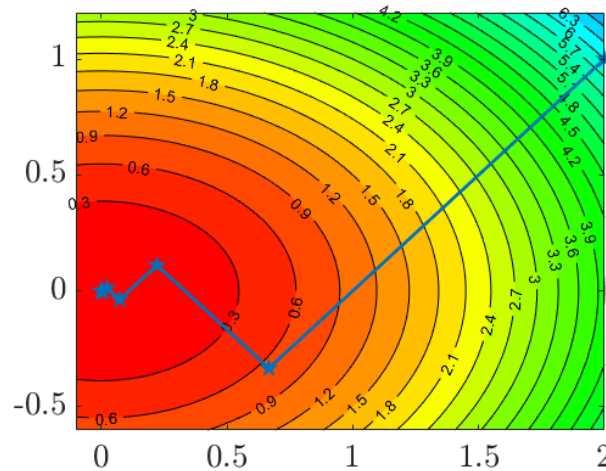


Figure 4: Gradient descent with exact line search for $f(\mathbf{x}) = x_1^2 + 2x_2^2$. After 13 iterations, the method converges to the optimum $(0, 0)$.

The Zig-Zag Effect

An evident behavior of the gradient method as illustrated in Figure 4 is the “zig-zag” effect, meaning that the direction found at the k -iteration $\mathbf{x}^{k+1} - \mathbf{x}^k$ is orthogonal to the direction found at the $(k + 1)$ -th iteration $\mathbf{x}^{k+2} - \mathbf{x}^{k+1}$. We now establish this result.

Lemma. Let $\{\mathbf{x}^k\}_{k>0}$ be the sequence generated by the gradient method with exact line



search for solving a problem of minimizing a continuously differentiable function f . Then for any $k = 0, 1, 2, \dots$

$$(\mathbf{x}^{k+2} - \mathbf{x}^{k+1})^\top (\mathbf{x}^{k+1} - \mathbf{x}^k) = 0.$$

Proof. First, we write $\mathbf{x}^{k+1} - \mathbf{x}^k = -t^k \nabla f(\mathbf{x}^k)$, and $\mathbf{x}^{k+2} - \mathbf{x}^{k+1} = -t^{k+1} \nabla f(\mathbf{x}^{k+1})$. Therefore, we need to prove that $\nabla f(\mathbf{x}^k)^\top \nabla f(\mathbf{x}^{k+1}) = 0$. The exact line search is by definition

$$t^k \in \operatorname{argmin}_{t \geq 0} \left\{ g(t) \equiv f(\mathbf{x}^k - t \nabla f(\mathbf{x}^k)) \right\}$$

Hence,

$$\begin{aligned} g'(t^k) &= 0, \\ -\nabla f(\mathbf{x}^k)^\top \nabla f(\mathbf{x}^k - t^k \nabla f(\mathbf{x}^k)) &= 0, \\ -\nabla f(\mathbf{x}^k)^\top \nabla f(\mathbf{x}^{k+1}) &= 0. \end{aligned}$$

□

Convergence of the Gradient Method

We begin this discussion with a computational example.

Example. Solve

$$\min x_1^2 + 2x_2^2$$

using gradient descent with constant stepsize. Set $\mathbf{x}^0 = (2, 1)$, $\varepsilon = 10^{-5}$, and $\bar{t} = 0.1$. The MATLAB code for this method can be found the end of the document. The iteration sequence reads

iter_number = 1	norm_grad = 4.000000	fun_val = 3.280000
iter_number = 2	norm_grad = 2.937210	fun_val = 1.897600
⋮	⋮	⋮
iter_number = 3	norm_grad = 2.222791	fun_val = 1.141888
iter_number = 56	norm_grad = 0.000015	fun_val = 0.000000
iter_number = 57	norm_grad = 0.000012	fun_val = 0.000000
iter_number = 58	norm_grad = 0.000010	fun_val = 0.000000

achieving convergence after 58 iterations. If we increase the stepsize parameter to $\bar{t} = 10$, we observe the history below



iter_number = 1	norm_grad = 1783.488716	fun_val = 476806.000000
iter_number = 2	norm_grad = 656209.693339	fun_val = 56962873606.00
iter_number = 3	norm_grad = 256032703.004797	fun_val = 8318300807
⋮	⋮	⋮
iter_number = 119	norm_grad = NaN	fun_val = NaN

In this case, the sequence diverges. This leads us to a very important question: how can we choose the constant stepsize so that convergence is guaranteed?

Lipschitz Continuity of the Gradient

Definition (Lipschitz Gradient). Let f be a continuously differentiable function over $\mathbb{R}^n$. We say that f has a Lipschitz gradient if there exists $L \geq 0$ for which

$$\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\| \leq L\|\mathbf{x} - \mathbf{y}\|,$$

for any $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$. L is called the **Lipschitz constant**.

Some relevant remarks:

- If ∇f is Lipschitz with constant L , then it is also Lipschitz with constant $\tilde{L}$ for all $\tilde{L} \geq L$.
- The class of functions with Lipschitz gradient with constant L is denoted by $C_L^{1,1}(\mathbb{R}^n)$ or just $C_L^{1,1}$. When the constant is not relevant, we simply denote the class by $C^{1,1}$.
- **Linear functions** - Given $\mathbf{a} \in \mathbb{R}^n$, the function $f(\mathbf{x}) = \mathbf{a}^\top \mathbf{x}$ is in $C_0^{1,1}$.
- **Quadratic functions** - Let $\mathbf{A}$ be a symmetric $n \times n$ matrix, $\mathbf{b} \in \mathbb{R}^n$ and $c \in \mathbb{R}$. Then the function $f(\mathbf{x}) = \mathbf{x}^\top \mathbf{A} \mathbf{x} + 2\mathbf{b}^\top \mathbf{x} + c$ is a $C^{1,1}$ function. The smallest Lipschitz constant of ∇f is $2\|\mathbf{A}\|_2$ (why?).

Theorem (Equivalence to Boundedness of the Hessian). Let f be a twice continuously differentiable function over $\mathbb{R}^n$. Then the following two claims are equivalent:

1. $f \in C_L^{1,1}(\mathbb{R}^n)$.
2. $\|\nabla^2 f(\mathbf{x})\| \leq L$ for any $\mathbf{x} \in \mathbb{R}^n$.

Proof. (2) $\Rightarrow$ (1). Suppose that $\|\nabla^2 f(\mathbf{x})\| \leq L$ for any $\mathbf{x} \in \mathbb{R}^n$. Then by the fundamental theorem of calculus we have for all $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$

$$\begin{aligned} \nabla f(\mathbf{y}) &= \nabla f(\mathbf{x}) + \int_0^1 \nabla^2 f(\mathbf{x} + t(\mathbf{y} - \mathbf{x}))(\mathbf{y} - \mathbf{x}) dt \\ &= \nabla f(\mathbf{x}) + \left(\int_0^1 \nabla^2 f(\mathbf{x} + t(\mathbf{y} - \mathbf{x})) dt \right) \cdot (\mathbf{y} - \mathbf{x}) \end{aligned}$$



Thus,

$$\begin{aligned}
 \|\nabla f(\mathbf{y}) - \nabla f(\mathbf{x})\| &= \left\| \left(\int_0^1 \nabla^2 f(\mathbf{x} + t(\mathbf{y} - \mathbf{x})) dt \right) \cdot (\mathbf{y} - \mathbf{x}) \right\| \\
 &\leq \left\| \int_0^1 \nabla^2 f(\mathbf{x} + t(\mathbf{y} - \mathbf{x})) dt \right\| \|\mathbf{y} - \mathbf{x}\| \\
 &\leq \left(\int_0^1 \|\nabla^2 f(\mathbf{x} + t(\mathbf{y} - \mathbf{x}))\| dt \right) \|\mathbf{y} - \mathbf{x}\| \\
 &\leq L \|\mathbf{y} - \mathbf{x}\|
 \end{aligned}$$

establishing the desired result $f \in C_L^{1,1}$.

Now we prove (1) $\Rightarrow$ (2). Suppose now that $f \in C_L^{1,1}$. Then by the fundamental theorem of calculus for any $\mathbf{d} \in \mathbb{R}^n$ and $\alpha > 0$ we have

$$\nabla f(\mathbf{x} + \alpha \mathbf{d}) - \nabla f(\mathbf{x}) = \int_0^\alpha \nabla^2 f(\mathbf{x} + t\mathbf{d}) \mathbf{d} dt$$

Thus,

$$\left\| \left(\int_0^\alpha \nabla^2 f(\mathbf{x} + t\mathbf{d}) dt \right) \mathbf{d} \right\| = \|\nabla f(\mathbf{x} + \alpha \mathbf{d}) - \nabla f(\mathbf{x})\| \leq \alpha L \|\mathbf{d}\|$$

Dividing by α and taking the limit $\alpha \rightarrow 0^+$, we obtain

$$\|\nabla^2 f(\mathbf{x}) \mathbf{d}\| \leq L \|\mathbf{d}\|$$

implying that $\|\nabla^2 f(\mathbf{x})\| \leq L$. □

Example. Show that $f(\mathbf{x}) = \sqrt{1 + x^2} \in C_L^{1,1}$.

Main Convergence Result

We now state an important result connecting functions in $C_L^{1,1}(\mathbb{R}^n)$ and gradient descent.

Lemma (Sufficient decrease of the gradient method). *Let $f \in C_L^{1,1}(\mathbb{R}^n)$. Let $\{\mathbf{x}^k\}_{k \geq 0}$ be the sequence generated by the gradient method for solving*

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x})$$

with one of the following stepsize strategies:

- constant stepsize $\bar{t} \in (0, \frac{2}{L})$,
- exact line search,
- backtracking procedure with parameters $s \in \mathbb{R}_{++}$, $\alpha \in (0, 1)$, and $\beta \in (0, 1)$.



Then

$$f(\mathbf{x}^k) - f(\mathbf{x}^{k+1}) \geq M \left\| \nabla f(\mathbf{x}^k) \right\|^2$$

where

$$M = \begin{cases} \bar{t} \left(1 - \frac{\bar{t}L}{2} \right) & \text{constant stepsize,} \\ \frac{1}{2L} & \text{exact line search,} \\ \alpha \min \left\{ s, \frac{2(1-\alpha)\beta}{L} \right\} & \text{backtracking.} \end{cases}$$

We will now show the convergence of the norms of the gradients $\left\| \nabla f(\mathbf{x}^k) \right\|$ to zero.

Theorem (Convergence of the Gradient Method). *Let $\{\mathbf{x}^k\}_{k \geq 0}$ be the sequence generated by the gradient descent method for solving*

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x})$$

with one of the following stepsize strategies:

- *constant stepsize $\bar{t} \in (0, \frac{2}{L})$,*
- *exact line search,*
- *backtracking procedure with parameters $s > 0$ and $\alpha, \beta \in (0, 1)$.*

Assume that

- *$f \in C_L^{1,1}(\mathbb{R}^n)$.*
- *f is bounded below over $\mathbb{R}^n$, that is, there exists $m \in \mathbb{R}$ such that $f(\mathbf{x}) > m$ for all $\mathbf{x} \in \mathbb{R}^n$.*

Then:

1. *for any k , $f(\mathbf{x}^{k+1}) < f(\mathbf{x}^k)$ unless $\nabla f(\mathbf{x}^k) = 0$.*
2. *$\nabla f(\mathbf{x}^k) \rightarrow 0$ as $k \rightarrow \infty$.*

Proof. (1) By the sufficient decrease of the gradient method (previous lemma) we have that

$$f(\mathbf{x}^k) - f(\mathbf{x}^{k+1}) \geq M \left\| \nabla f(\mathbf{x}^k) \right\|^2 \geq 0$$

for some constant $M > 0$, and hence the equality $f(\mathbf{x}^k) = f(\mathbf{x}^{k+1})$ can hold only when $\nabla f(\mathbf{x}^k) = 0$.

(2) Since the sequence $\{f(\mathbf{x}^k)\}_{k \geq 0}$ is nonincreasing and bounded below, it converges. Thus, in particular $f(\mathbf{x}^k) - f(\mathbf{x}^{k+1}) \rightarrow 0$ as $k \rightarrow \infty$, which combined with the sufficient decrease of the gradient method implies that $\left\| \nabla f(\mathbf{x}^k) \right\| \rightarrow 0$ as $k \rightarrow \infty$. $\square$



Improved convergence result of the gradient method for convex functions

Based on the results of the previous section, if the objective function $f \in C_L^{1,1}(\mathbb{R}^n)$, and the stepsize is chosen to guarantee sufficient decrease then the sequence $\mathbf{x}^k$ generated by the algorithm satisfies

$$f(\mathbf{x}^k) - f(\mathbf{x}^{k+1}) \geq M \|\nabla f(\mathbf{x}^k)\|^2$$

where M is a constant which depends on the used stepsize rule.

Now, if we sum over k , from $0, \dots, N-1$, then

$$f(\mathbf{x}^0) - f(\mathbf{x}^N) \geq M \sum_{k=0}^{N-1} \|\nabla f(\mathbf{x}^k)\|^2 \quad (1)$$

$$\geq MN \min_{0 \leq k \leq N-1} \|\nabla f(\mathbf{x}^k)\|^2 \quad (2)$$

Moreover, since $f(\mathbf{x}^N) \geq f(\mathbf{x}^*)$, where $\mathbf{x}^*$ is the solution of the problem, the gradient method can find a point $\mathbf{x}^k$ such that

$$\min_{0 \leq k \leq N-1} \|\nabla f(\mathbf{x}^k)\|^2 \leq \sqrt{\frac{f(\mathbf{x}^0) - f(\mathbf{x}^*)}{MN}}$$

This relation tells us only that, after $N-1$ iterations, we will find a point $\mathbf{x}^k$ which is *nearly stationary*. However, this result can be improved under the additional assumption of convexity of the objective function. In fact, the following result holds

Theorem (Convergence of the gradient method for convex functions). *Let $f: C \subset \mathbb{R}^n \rightarrow \mathbb{R}$ be a convex function over the convex set C , and $f \in C_L^{1,1}(\mathbb{R}^n)$. Let $\mathbf{x}^* \in \mathbb{R}^n$ be the solution of the problem*

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x})$$

Then, the gradient method with step-length $t_k = 1/L$ generates a sequence of iterates $\{\mathbf{x}^k\}_{k \geq 0}$ that satisfies

$$f(\mathbf{x}^K) - f^* \leq \frac{L}{2K} \|\mathbf{x}^0 - \mathbf{x}^*\|^2 \quad K = 1, 2, \dots$$

with $f^ = f(\mathbf{x}^*)$.*

The Condition Number

Example. Implement our good old example

$$\min x_1^2 + 2x_2^2$$



using gradient descent with backtracking and parameters $\mathbf{x}^0 = (2, 1)$, $s = 2$, $\alpha = 0.25$, $\beta = 0.5$, and $\varepsilon = 10^{-5}$. Using the code at the end of the document you should see that it converges in only two iterations! However, if you try to solve

$$\min 0.01x_1^2 + x_2^2$$

with the same method and parameters, it will take about 200 iterations to reach the optimum $(0, 0)$. Can we detect key properties of the objective function that imply fast/slow convergence? We will see that in the quadratic case, a fundamental characterization is given by the condition number of the associated quadratic form. We recall the definition of condition number.

Definition (Condition Number). *Let $\mathbf{A}$ be an $n \times n$ positive definite matrix. Then the condition number of $\mathbf{A}$ is defined by*

$$\kappa(\mathbf{A}) = \frac{\lambda_{\max}(\mathbf{A})}{\lambda_{\min}(\mathbf{A})}$$

where $\lambda_{\max}(\mathbf{A})$ and $\lambda_{\min}(\mathbf{A})$ are the largest and smaller eigenvalues, respectively.

Matrices (or quadratic functions) with large condition number are called ill-conditioned. Matrices with small condition number are called well-conditioned. Among other things, the condition number gives an idea of the error amplification when working with the matrix $\mathbf{A}$. For an ill-conditioned matrix, small perturbations in the matrix entries can lead to large errors when solving a linear system, or computing $\mathbf{A}^{-1}$. We continue with a technical lemma.

Lemma (Kantorovich Inequality). *Let $\mathbf{A}$ be a positive definite $n \times n$ matrix. Then for any $0 \neq \mathbf{x} \in \mathbb{R}^n$ the inequality*

$$\frac{(\mathbf{x}^\top \mathbf{x})^2}{(\mathbf{x}^\top \mathbf{A} \mathbf{x})(\mathbf{x}^\top \mathbf{A}^{-1} \mathbf{x})} \geq \frac{4\lambda_{\max}(\mathbf{A})\lambda_{\min}(\mathbf{A})}{(\lambda_{\max}(\mathbf{A}) + \lambda_{\min}(\mathbf{A}))^2}$$

holds.

We are now in position to state a precise result regarding the minimization of quadratic functions via gradient descent and its rate of convergence based on the condition number of the matrix $\mathbf{A}$.

Theorem (Gradient Method for Minimizing $\mathbf{x}^\top \mathbf{A} \mathbf{x}$). *Let $\{\mathbf{x}^k\}_{k \geq 0}$ be the sequence generated by the gradient method with exact line search for solving the problem*

$$\min_{\mathbf{x} \in \mathbb{R}^n} \mathbf{x}^\top \mathbf{A} \mathbf{x} \quad (\mathbf{A} > 0),$$

Then for any $k = 0, 1, \dots$

$$f(\mathbf{x}^{k+1}) \leq \left(\frac{M - m}{M + m} \right)^2 f(\mathbf{x}^k),$$

where $M = \lambda_{\max}(\mathbf{A})$, and $m = \lambda_{\min}(\mathbf{A})$.



Proof. The gradient descent iteration with exact line search for a quadratic function reads $\mathbf{x}^{k+1} = \mathbf{x}^k - t^k \mathbf{d}^k$, where $t^k = \frac{(\mathbf{d}^k)^\top \mathbf{d}^k}{2(\mathbf{d}^k)^\top \mathbf{A} \mathbf{d}^k}$, and $\mathbf{d}^k = 2\mathbf{A}\mathbf{x}^k$.

Plugging in the expression for t^k

$$\begin{aligned} f(\mathbf{x}^{k+1}) &= (\mathbf{x}^k)^\top \mathbf{A} \mathbf{x}^k - \frac{1}{4} \frac{((\mathbf{d}^k)^\top \mathbf{d}^k)^2}{(\mathbf{d}^k)^\top \mathbf{A} \mathbf{d}^k} \\ &= (\mathbf{x}^k)^\top \mathbf{A} \mathbf{x}^k \left(1 - \frac{1}{4} \frac{((\mathbf{d}^k)^\top \mathbf{d}^k)^2}{((\mathbf{d}^k)^\top \mathbf{A} \mathbf{d}^k) ((\mathbf{x}^k)^\top \mathbf{A} \mathbf{A}^{-1} \mathbf{A} \mathbf{x}^k)} \right) \\ &= \left(1 - \frac{((\mathbf{d}^k)^\top \mathbf{d}^k)^2}{((\mathbf{d}^k)^\top \mathbf{A} \mathbf{d}^k) ((\mathbf{d}^k)^\top \mathbf{A}^{-1} \mathbf{d}^k)} \right) f(\mathbf{x}^k). \end{aligned}$$

Finally, using Kantorovich's Inequality:

$$f(\mathbf{x}^{k+1}) \leq \left(1 - \frac{4Mm}{(M+m)^2} \right) f(\mathbf{x}^k) = \left(\frac{M-m}{M+m} \right)^2 f(\mathbf{x}^k) = \left(\frac{\kappa(\mathbf{A}) - 1}{\kappa(\mathbf{A}) + 1} \right)^2 f(\mathbf{x}^k)$$

□

From the expression above, we deduce that large condition number implies a large number of iterations of the gradient method. A small condition number (slightly greater than 1) implies a small number of iterations of the gradient method. For a non-quadratic function, the asymptotic rate of convergence of $\mathbf{x}^k$ to a stationary point $\mathbf{x}^*$ is usually determined by the condition number of $\nabla^2 f(\mathbf{x}^*)$.

Example A severely ill-conditioned function is the so called Rosenbrock function:

$$\min \left\{ f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2 \right\}$$

The optimal solution to this problem is easily found as $(x_1, x_2) = (1, 1)$, with optimal value 0. The gradient is given by

$$\nabla f(\mathbf{x}) = \begin{pmatrix} -400x_1(x_2 - x_1^2) - 2(1 - x_1) \\ 200(x_2 - x_1^2) \end{pmatrix},$$

and the Hessian

$$\nabla^2 f(\mathbf{x}) = \begin{pmatrix} -400x_2 + 1200x_1^2 + 2 & -400x_1 \\ -400x_1 & 200 \end{pmatrix}.$$

Evaluating the Hessian at the optimum we obtain

$$\nabla^2 f(1, 1) = \begin{pmatrix} 802 & -400 \\ -400 & 200 \end{pmatrix},$$

with a high condition number: 2508. Solving the Rosenbrock problem with gradient descent and backtracking stepsize selection and parameters $\mathbf{x}^0 = (2, 5)$, $s = 2$, $\alpha = 0.25$, $\beta = 0.5$, $\varepsilon = 10^{-5}$, leads to 6890(!) iterations. Figure 5 depicts the slow convergence due to poor conditioning of the Hessian around the optimum.



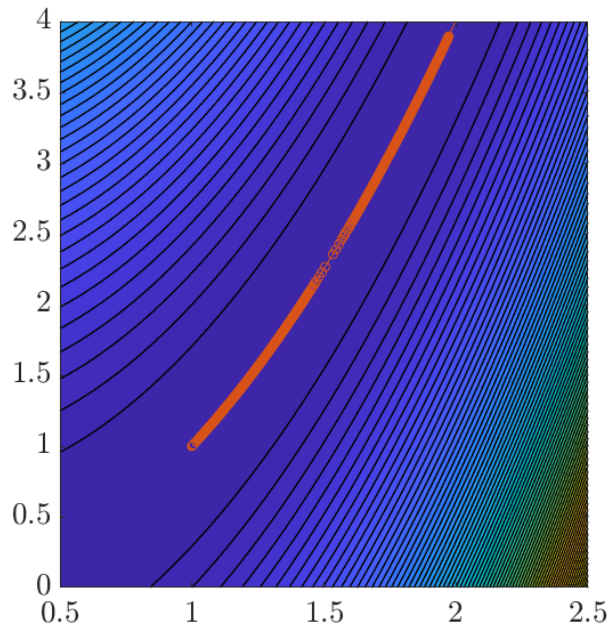


Figure 5: Gradient descent with backtracking line search for the Rosenbrock problem. Convergence towards the optimum (1,1) is achieved after several thousands iterations. The method is extremely slow due to poor conditioning of the Hessian around the optimum.

Scaled Gradient Method

A way to mitigate the slow convergence due to poor conditioning of the Hessian is to formulate a rescaled version of the problem. Consider the minimization problem

$$\min \{f(\mathbf{x}) : \mathbf{x} \in \mathbb{R}^n\}$$

For a given nonsingular matrix $\mathbf{S} \in \mathbb{R}^{n \times n}$, we make the linear change of variables $\mathbf{x} = \mathbf{S}\mathbf{y}$, and obtain the equivalent problem

$$\min \{g(\mathbf{y}) \equiv f(\mathbf{S}\mathbf{y}) : \mathbf{y} \in \mathbb{R}^n\} .$$

Since $\nabla g(\mathbf{y}) = \mathbf{S}^\top \nabla f(\mathbf{S}\mathbf{y}) = \mathbf{S}^\top \nabla f(\mathbf{x})$, the gradient method for the rescaled problem reads

$$\mathbf{y}^{k+1} = \mathbf{y}^k - t^k \mathbf{S}^\top \nabla f(\mathbf{S}\mathbf{y}^k) .$$

Multiplying the latter equality by $\mathbf{S}$ from the left, and using the notation $\mathbf{x}^k = \mathbf{S}\mathbf{y}^k$

$$\mathbf{x}^{k+1} = \mathbf{x}^k - t^k \mathbf{S} \mathbf{S}^\top \nabla f(\mathbf{x}^k)$$

Defining $\mathbf{D} = \mathbf{S} \mathbf{S}^\top$, we obtain the scaled gradient method:

$$\mathbf{x}^{k+1} = \mathbf{x}^k - t^k \mathbf{D} \nabla f(\mathbf{x}^k) .$$



Note that $\mathbf{D} \succ \mathbf{0}$, so the direction $-\mathbf{D}\nabla f(\mathbf{x}^k)$ is a descent direction:

$$f'(\mathbf{x}^k; -\mathbf{D}\nabla f(\mathbf{x}^k)) = -\nabla f(\mathbf{x}^k)^\top \mathbf{D}\nabla f(\mathbf{x}^k) < 0.$$

We also allow different scaling matrices at each iteration.

Algorithm 3: Scaled Gradient Method

Initialization: A tolerance parameter $\varepsilon > 0$ and $\mathbf{x}^0 \in \mathbb{R}^n$.

General Step: for any $k = 0, 1, 2, \dots$ execute the following steps:

- 1 Pick a scaling matrix $\mathbf{D}^k \succ \mathbf{0}$
- 2 Pick a stepsize t^k by a line search procedure on the function

$$g(t) = f(\mathbf{x}^k - t\mathbf{D}^k\nabla f(\mathbf{x}^k))$$

- 3 Set $\mathbf{x}^{k+1} = \mathbf{x}^k - t^k\mathbf{D}^k\nabla f(\mathbf{x}^k)$.
 - 4 If $\|\nabla f(\mathbf{x}^{k+1})\| \leq \varepsilon$, then STOP and $\mathbf{x}^{k+1}$ is the output.
-

Choosing the Scaling Matrix $\mathbf{D}^k$. The scaled gradient method with scaling matrix $\mathbf{D}$ is equivalent to the gradient method employed on the function $g(\mathbf{y}) = f(\mathbf{D}^{1/2}\mathbf{y})$, where $\mathbf{D}^{1/2}$ is a matrix such that $\mathbf{D}^{1/2}\mathbf{D}^{1/2} = \mathbf{D}$. Note that the gradient and Hessian of g are given by

$$\begin{aligned}\nabla g(\mathbf{y}) &= \mathbf{D}^{1/2} \nabla f(\mathbf{D}^{1/2}\mathbf{y}) = \mathbf{D}^{1/2} \nabla f(\mathbf{x}) \\ \nabla^2 g(\mathbf{y}) &= \mathbf{D}^{1/2} \nabla^2 f(\mathbf{D}^{1/2}\mathbf{y}) \mathbf{D}^{1/2} = \mathbf{D}^{1/2} \nabla^2 f(\mathbf{x}) \mathbf{D}^{1/2}\end{aligned}$$

The objective is usually to pick $\mathbf{D}^k$ so as to make $(\mathbf{D}^k)^{1/2} \nabla^2 f(\mathbf{x}^k) (\mathbf{D}^k)^{1/2}$ as well-conditioned as possible. A well known choice (Newton's method) is to pick $\mathbf{D}^k = (\nabla^2 f(\mathbf{x}^k))^{-1}$. Another alternative is to use a diagonal scaling: $\mathbf{D}^k$ is picked to be diagonal. For example,

$$(\mathbf{D}^k)_{ii} = \left(\frac{\partial^2 f(\mathbf{x}^k)}{\partial x_i^2} \right)^{-1}$$

Using diagonal scaling can be very effective when the decision variables are of different magnitudes.

Example Revisit the Rosenbrock example (p. 12) using a suitable scaling.



```

1  function [hist,x,fun_val]=gradient_method_quadratic(A,b,x0,epsilon);
2  % INPUT
3  % =====
4  % A ..... the positive definite matrix associated with the ...
   objective function
5  % b ..... a column vector associated with the linear part of ...
   the objective function
6  % x0 ..... starting point of the method
7  % epsilon . tolerance parameter
8  % OUTPUT
9  % =====
10 % hist.....convergence history of the iterations
11 % x ..... an optimal solution (up to a tolerance) of ...
   min(x^{\top} A x+2 b^{\top} x)
12 % fun_val . the optimal function value up to a tolerance
13
14
15 x=x0;
16 iter=0;
17 grad=2*(A*x+b);
18 hist=x0;
19 while (norm(grad)>epsilon)
20   iter=iter+1;
21   t=norm(grad)^2/(2*grad'*A*grad);
22   x=x-t*grad;
23   grad=2*(A*x+b);
24   fun_val=x'*A*x+2*b'*x;
25   hist=[hist x];
26   fprintf('iter_number = %3d norm_grad = %2.6f fun_val = %2.6f ...
   \n',iter,norm(grad),fun_val);
27 end

```




```

1 function [hist,x,fun_val]=gradient_method_constant(f,g,x0,t,epsilon)
2 % Gradient method with constant stepsize
3 %
4 % INPUT
5 %=====
6 % f ..... objective function
7 % g ..... gradient of the objective function
8 % x0..... initial point
9 % t ..... constant stepsize
10 % epsilon ... tolerance parameter
11 % OUTPUT
12 %=====
13 % x ..... optimsl solution (up to a tolerance)
14 % of min f(x)
15 % fun_val ... optimal function value
16 x=x0;
17 grad=g(x);
18 iter=0;
19 hist=x0;
20 while (norm(grad)>epsilon)
21 iter=iter+1;
22 x=x-t*grad;
23 fun_val=f(x);
24 grad=g(x);
25 hist=[hist x];
26 fprintf('iter_number = %3d norm_grad = %2.6f fun_val = %2.6f ...
    \n',iter,norm(grad),fun_val);
27 end

```



```

1 function ...
    [x,fun_val]=gradient_method_backtracking(f,g,x0,s,alpha,beta,epsilon)
2 % Gradient method with backtracking stepsize rule
3 %
4 % INPUT
5 %=====
6 % f ..... objective function
7 % g ..... gradient of the objective function
8 % x0..... initial point
9 % s ..... initial choice of stepsize
10 % alpha ..... tolerance parameter for the stepsize selection
11 % beta ..... the constant in which the stepsize is multiplied
12 % at each backtracking step (0<beta<1)
13 % epsilon ... tolerance parameter for stopping rule
14 % OUTPUT
15 %=====
16 % x ..... optimal solution (up to a tolerance)
17 % of min f(x)
18 % fun_val ... optimal function value
19 x=x0;
20 grad=g(x);
21 fun_val=f(x);
22 iter=0;
23 while (norm(grad)>epsilon)
24     iter=iter+1;
25     t=s;
26     while (fun_val-f(x-t*grad)<alpha*t*norm(grad)^2)
27         t=beta*t;
28     end
29     x=x-t*grad;
30     fun_val=f(x);
31     grad=g(x);
32     fprintf('iter_number = %3d norm_grad = %2.6f fun_val = %2.6f ...
        \n',iter,norm(grad),fun_val);
33 end

```

The Gauss-Newton Method

We use the gradient method from last week to build an algorithm for solving nonlinear least squares problem of the type:

$$(\text{NLS}): \min_{\mathbf{x} \in \mathbb{R}^n} \left\{ g(\mathbf{x}) \equiv \sum_{i=1}^m (f_i(\mathbf{x}) - c_i)^2 \right\}$$



$f_1, \dots, f_m$ are continuously differentiable over $\mathbb{R}^n$ and $c_1, \dots, c_m \in \mathbb{R}$.

Denote:

$$F(\mathbf{x}) = \begin{pmatrix} f_1(\mathbf{x}) - c_1 \\ f_2(\mathbf{x}) - c_2 \\ \vdots \\ f_m(\mathbf{x}) - c_m \end{pmatrix}$$

Then the problem becomes:

$$\min_{\mathbf{x} \in \mathbb{R}^n} \|F(\mathbf{x})\|^2$$



Figure 6: Sir Isaac Newton (1642–1727) the protagonist of this week, needs no introduction. On the left, he discovers Pink Floyd’s “The Dark Side of the Moon” album. On the right, a recreation of a little domestic incident involving fire and his notes on Optics.

The Gauss-Newton method is an algorithm for solving this particular problem, and it reads as follows. Given the k th iterate $\mathbf{x}^k$, the next iterate is chosen to minimize the sum of squares of the linearized terms, that is,

$$\mathbf{x}^{k+1} = \operatorname{argmin}_{\mathbf{x} \in \mathbb{R}^n} \left\{ \sum_{i=1}^m \left[f_i(\mathbf{x}^k) + \nabla f_i(\mathbf{x}^k)^\top (\mathbf{x} - \mathbf{x}^k) - c_i \right]^2 \right\}$$

The general step actually consists of solving the linear LS problem

$$\min \left\| \mathbf{A}^k \mathbf{x} - \mathbf{b}^k \right\|^2$$

where

$$\mathbf{A}^k = \begin{pmatrix} \nabla f_1(\mathbf{x}^k)^\top \\ \nabla f_2(\mathbf{x}^k)^\top \\ \vdots \\ \nabla f_m(\mathbf{x}^k)^\top \end{pmatrix} = J(\mathbf{x}^k),$$



is the Jacobian matrix, assumed to have full column rank, and

$$\mathbf{b}^k = \begin{pmatrix} \nabla f_1(\mathbf{x}^k)^\top \mathbf{x}^k - f_1(\mathbf{x}^k) + c_1 \\ \nabla f_2(\mathbf{x}^k)^\top \mathbf{x}^k - f_2(\mathbf{x}^k) + c_2 \\ \vdots \\ \nabla f_m(\mathbf{x}^k)^\top \mathbf{x}^k - f_m(\mathbf{x}^k) + c_m \end{pmatrix} = J(\mathbf{x}^k)\mathbf{x}^k - F(\mathbf{x}^k).$$

The Gauss-Newton method can thus be written as:

$$\mathbf{x}^{k+1} = \left(J(\mathbf{x}^k)^\top J(\mathbf{x}^k) \right)^{-1} J(\mathbf{x}^k)^\top \mathbf{b}^k.$$

Note that the gradient of the objective function $g(\mathbf{x}) = \|F(\mathbf{x})\|^2$ is

$$\nabla g(\mathbf{x}) = 2J(\mathbf{x})^\top F(\mathbf{x}).$$

The Gauss-Newton method can be rewritten as follows:

$$\begin{aligned} \mathbf{x}^{k+1} &= \left(J(\mathbf{x}^k)^\top J(\mathbf{x}^k) \right)^{-1} J(\mathbf{x}^k)^\top \left(J(\mathbf{x}^k)\mathbf{x}^k - F(\mathbf{x}^k) \right) \\ &= \mathbf{x}^k - \left(J(\mathbf{x}^k)^\top J(\mathbf{x}^k) \right)^{-1} J(\mathbf{x}^k)^\top F(\mathbf{x}^k) \\ &= \mathbf{x}^k - \frac{1}{2} \left(J(\mathbf{x}^k)^\top J(\mathbf{x}^k) \right)^{-1} \nabla g(\mathbf{x}^k) \end{aligned}$$

that is, it is a scaled gradient method with a special choice of scaling matrix:

$$\mathbf{D}^k = \frac{1}{2} \left(J(\mathbf{x}^k)^\top J(\mathbf{x}^k) \right)^{-1}.$$

The Damped Gauss-Newton Method

The Gauss-Newton method does not incorporate a stepsize, which might cause it to diverge. A well known variation of the method incorporating stepsizes is the **damped Gauss-Newton method**.

Algorithm 4: The Damped Gauss-Newton Method

Initialization: A tolerance parameter $\varepsilon > 0$ and $\mathbf{x}^0 \in \mathbb{R}^n$.

General Step: for any $k = 0, 1, 2, \dots$ execute the following steps:

- 1 Set $\mathbf{d}^k = - \left(J(\mathbf{x}^k)^\top J(\mathbf{x}^k) \right)^{-1} J(\mathbf{x}^k)^\top F(\mathbf{x}^k)$.
- 2 Set t^k by a line search procedure on the function

$$h(t) = g(\mathbf{x}^k + t\mathbf{d}^k).$$

- 3 Set $\mathbf{x}^{k+1} = \mathbf{x}^k + t^k \mathbf{d}^k$.
 - 4 If $\|\nabla g(\mathbf{x}^{k+1})\| \leq \varepsilon$, then STOP and $\mathbf{x}^{k+1}$ is the output.
-



The Fermat-Weber Problem and Weiszfeld's Method

The Fermat-Weber problem is stated as follows. Given m points in $\mathbb{R}^n : \mathbf{a}_1, \dots, \mathbf{a}_m$, also called *anchor points*, and m weights $\omega_1, \omega_2, \dots, \omega_m > 0$, find a point $\mathbf{x} \in \mathbb{R}^n$ that minimizes the weighted distance of $\mathbf{x}$ to each of the points $\omega_1, \omega_2, \dots, \omega_m > 0$, that is

$$\min_{\mathbf{x} \in \mathbb{R}^n} \left\{ f(\mathbf{x}) \equiv \sum_{i=1}^m \omega_i \|\mathbf{x} - \mathbf{a}_i\| \right\}$$

Note that the objective function is not differentiable at the anchor points $\mathbf{a}_1, \dots, \mathbf{a}_m$ (why?).

In terms of applications, this is one of the simplest instances of facility location problems. In 1937, with only 16 years old, the Hungarian mathematician Endre Weiszfeld proposed a method for solving this problem, unsurprisingly known as *Weiszfeld's Method*. Under the assumption that the optimum $\mathbf{x}$ is not an anchor point, we write the stationarity condition

$$\begin{aligned} \nabla f(\mathbf{x}) &= 0, \\ \sum_{i=1}^m \omega_i \frac{\mathbf{x} - \mathbf{a}_i}{\|\mathbf{x} - \mathbf{a}_i\|} &= 0, \\ \left(\sum_{i=1}^m \frac{\omega_i}{\|\mathbf{x} - \mathbf{a}_i\|} \right) \mathbf{x} &= \sum_{i=1}^m \frac{\omega_i \mathbf{a}_i}{\|\mathbf{x} - \mathbf{a}_i\|}. \end{aligned}$$

The stationarity condition can be written as a fixed point $\mathbf{x} = T(\mathbf{x})$, where T is the operator

$$T(\mathbf{x}) \equiv \frac{1}{\sum_{i=1}^m \frac{\omega_i}{\|\mathbf{x} - \mathbf{a}_i\|}} \sum_{i=1}^m \frac{\omega_i \mathbf{a}_i}{\|\mathbf{x} - \mathbf{a}_i\|}.$$

Weiszfeld's method is the fixed point iteration:

$$\mathbf{x}^{k+1} = T(\mathbf{x}^k),$$

which can be interpreted as a gradient method since

$$\begin{aligned} \mathbf{x}^{k+1} &= \frac{1}{\sum_{i=1}^m \frac{\omega_i}{\|\mathbf{x}^k - \mathbf{a}_i\|}} \sum_{i=1}^m \frac{\omega_i \mathbf{a}_i}{\|\mathbf{x}^k - \mathbf{a}_i\|} \\ &= \mathbf{x}^k - \frac{1}{\sum_{i=1}^m \frac{1}{\|\mathbf{x}^k - \mathbf{a}_i\|}} \sum_{i=1}^m \omega_i \frac{\mathbf{x}^k - \mathbf{a}_i}{\|\mathbf{x}^k - \mathbf{a}_i\|} \\ &= \mathbf{x}^k - \frac{1}{\sum_{i=1}^m \frac{\omega_i}{\|\mathbf{x}^k - \mathbf{a}_i\|}} \nabla f(\mathbf{x}^k). \end{aligned}$$

Therefore, it corresponds to a gradient method with a special choice of stepsize:

$$t^k = \frac{1}{\sum_{i=1}^m \frac{\omega_i}{\|\mathbf{x}^k - \mathbf{a}_i\|}}$$



Newton's Method

We now discuss a different class of algorithms for solving the problem

$$\min \{f(\mathbf{x}) : \mathbf{x} \in \mathbb{R}^n\} .$$

Assuming that f is twice continuously differentiable over $\mathbb{R}^n$, looking for the optimum begins with the search of a stationary point $\mathbf{x}^*$ such that $\nabla f(\mathbf{x}^*) = 0$. This can be framed as finding a root (zero) for $g(\mathbf{x}) \equiv \nabla f(\mathbf{x})$. A classical algorithm for finding the zeros of a function is Newton's method. For $g : \mathbb{R} \rightarrow \mathbb{R}$ it reads

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \frac{g(\mathbf{x}^k)}{g'(\mathbf{x}^k)} .$$

We will utilise this idea for minimizing a function in several variables. In particular, going back to $f(\mathbf{x})$ and assuming it is twice continuously differentiable over $\mathbb{R}^n$, we generate a sequence $\{\mathbf{x}^k\}_{k=1}^{\infty}$ converging to a stationary point of $f(\mathbf{x})$ through the iteration

$$\mathbf{x}^{k+1} = \operatorname{argmin}_{\mathbf{x} \in \mathbb{R}^n} \left\{ f(\mathbf{x}^k) + \nabla f(\mathbf{x}^k)^\top (\mathbf{x} - \mathbf{x}^k) + \frac{1}{2} (\mathbf{x} - \mathbf{x}^k)^\top \nabla^2 f(\mathbf{x}^k) (\mathbf{x} - \mathbf{x}^k) \right\} .$$

This expression is interpreted as follows. Given a current value $\mathbf{x}^k$, we built a quadratic approximation of $f(\mathbf{x})$ around $\mathbf{x}^k$ (recall the quadratic approximation theorem), and find the minimizer of this parabolic approximation. In the case the minimizer exists, this is the next point of our sequence $\mathbf{x}^{k+1}$, where we move and repeat. This minimization problem not well-defined in general. However, if the Hessian is positive definite at $\mathbf{x}^k$, $\nabla^2 f(\mathbf{x}^k) > 0$, then the update reads

$$\mathbf{x}^{k+1} = \mathbf{x}^k - (\nabla^2 f(\mathbf{x}^k))^{-1} \nabla f(\mathbf{x}^k)$$

The vector

$$-(\nabla^2 f(\mathbf{x}^k))^{-1} \nabla f(\mathbf{x}^k)$$

is called Newton's direction. This algorithm is called Pure Newton's Method and it is summarized below.

Algorithm 5: Pure Newton's Method

Initialization: A tolerance parameter $\varepsilon > 0$ and $\mathbf{x}^0 \in \mathbb{R}^n$.

General Step: for any $k = 0, 1, 2, \dots$ execute the following steps:

- 1 Compute the Newton direction $\mathbf{d}^k$, which is the solution to the linear system

$$\nabla^2 f(\mathbf{x}^k) \mathbf{d}^k = -\nabla f(\mathbf{x}^k) .$$

- 2 Set $\mathbf{x}^{k+1} = \mathbf{x}^k + \mathbf{d}^k$.
 - 3 if $\|\nabla f(\mathbf{x}^{k+1})\| \leq \varepsilon$, then STOP and $\mathbf{x}^{k+1}$ is the output.
-



Convergence of Newton's Method

At the very least, Newton's method requires that $\nabla^2 f(\mathbf{x}) > 0$ for every $\mathbf{x} \in \mathbb{R}^n$, which in particular implies that there exists a unique optimal solution $\mathbf{x}^*$. However, this is not enough to guarantee convergence.

Example. Analyse what happens when $f(x) = \sqrt{1+x^2}$.

A lot of assumptions are required to be made in order to guarantee convergence of the method. However, Newton's method does have one very attractive feature. Under certain assumptions one can prove local quadratic rate of convergence, which means that near the optimal solution the errors $e^k = \|\mathbf{x}^k - \mathbf{x}^*\|$ satisfy an inequality $e^{k+1} \leq M(e^k)^2$ for some positive $M > 0$. This property essentially means that the number of accuracy digits is doubled at each iteration. This is in contrast to the gradient method in which the convergence theorems are rather independent in the starting point, but only "relatively" slow linear convergence is assured.

Theorem (Quadratic Local Convergence of Newton's Method). *Let f be a twice continuously differentiable function defined over $\mathbb{R}^n$. Assume that*

- *there exists $m > 0$ for which $\nabla^2 f(\mathbf{x}) \geq m\mathbf{I}$, for any $\mathbf{x} \in \mathbb{R}^n$,*
- *there exists $L > 0$ for which $\|\nabla^2 f(\mathbf{x}) - \nabla^2 f(\mathbf{y})\| \leq L\|\mathbf{x} - \mathbf{y}\|$ for any $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$.*

Let $\{\mathbf{x}^k\}_{k \geq 0}$ be the sequence generated by Newton's method and let $\mathbf{x}^$ be the unique minimizer of f over $\mathbb{R}^n$. Then for any $k = 0, 1, \dots$ the inequality*

$$\|\mathbf{x}^{k+1} - \mathbf{x}^*\| \leq \frac{L}{2m} \|\mathbf{x}^k - \mathbf{x}^*\|^2$$

holds. In addition, if $\|\mathbf{x}^0 - \mathbf{x}^\| \leq \frac{m}{L}$, then:*

$$\|\mathbf{x}^k - \mathbf{x}^*\| \leq \frac{2m}{L} \left(\frac{1}{2}\right)^{2^k}, \quad k = 0, 1, 2, \dots$$

Proof. We prove the first part of the result. Let k be a nonnegative integer. Then

$$\begin{aligned} \mathbf{x}^{k+1} - \mathbf{x}^* &= \mathbf{x}^k - (\nabla^2 f(\mathbf{x}^k))^{-1} \nabla f(\mathbf{x}^k) - \mathbf{x}^* \\ &= \mathbf{x}^k - \mathbf{x}^* + (\nabla^2 f(\mathbf{x}^k))^{-1} (\nabla f(\mathbf{x}^*) - \nabla f(\mathbf{x}^k)) \\ &= \mathbf{x}^k - \mathbf{x}^* + (\nabla^2 f(\mathbf{x}^k))^{-1} \int_0^1 \left[\nabla^2 f(\mathbf{x}^k + t(\mathbf{x}^* - \mathbf{x}^k)) \right] (\mathbf{x}^* - \mathbf{x}^k) dt \\ &= (\nabla^2 f(\mathbf{x}^k))^{-1} \int_0^1 \left[\nabla^2 f(\mathbf{x}^k + t(\mathbf{x}^* - \mathbf{x}^k)) - \nabla^2 f(\mathbf{x}^k) \right] (\mathbf{x}^* - \mathbf{x}^k) dt \end{aligned}$$

Combining the latter equality with the fact that $\nabla^2 f(\mathbf{x}^k) \geq m\mathbf{I}$, implies that

$$\|(\nabla^2 f(\mathbf{x}^k))^{-1}\| \leq \frac{1}{m},$$



Hence,

$$\begin{aligned}
\|\mathbf{x}^{k+1} - \mathbf{x}^*\| &\leq \|(\nabla^2 f(\mathbf{x}^k))^{-1}\| \left\| \int_0^1 \left[\nabla^2 f(\mathbf{x}^k + t(\mathbf{x}^* - \mathbf{x}^k)) - \nabla^2 f(\mathbf{x}^k) \right] (\mathbf{x}^* - \mathbf{x}^k) dt \right\| \\
&\leq \|(\nabla^2 f(\mathbf{x}^k))^{-1}\| \int_0^1 \left\| \left[\nabla^2 f(\mathbf{x}^k + t(\mathbf{x}^* - \mathbf{x}^k)) - \nabla^2 f(\mathbf{x}^k) \right] (\mathbf{x}^* - \mathbf{x}^k) \right\| dt \\
&\leq \|(\nabla^2 f(\mathbf{x}^k))^{-1}\| \int_0^1 \left\| \nabla^2 f(\mathbf{x}^k + t(\mathbf{x}^* - \mathbf{x}^k)) - \nabla^2 f(\mathbf{x}^k) \right\| \cdot \|\mathbf{x}^* - \mathbf{x}^k\| dt \\
&\leq \frac{L}{m} \int_0^1 t \|\mathbf{x}^k - \mathbf{x}^*\|^2 dt = \frac{L}{2m} \|\mathbf{x}^k - \mathbf{x}^*\|^2.
\end{aligned}$$

The second part of the theorem can be shown by induction (try it!). □

Numerical Example. Consider the minimization problem

$$\min 100x^4 + 0.01y^4.$$

Compare Newton's method against a gradient descent with backtracking. Repeat for

$$\min \sqrt{x_1^2 + 1} + \sqrt{x_2^2 + 1}.$$

Note that in this case, the Hessian of the function is

$$\nabla^2 f(\mathbf{x}) = \begin{pmatrix} \frac{1}{(x_1^2+1)^{3/2}} & 0 \\ 0 & \frac{1}{(x_2^2+1)^{3/2}} \end{pmatrix} > 0$$

but there does not exist an $m > 0$ for which $\nabla^2 f(\mathbf{x}) \geq m\mathbf{I}$. In this case using pure Newton's method will diverge. An alternative is to use a damped version of Newton's method, using backtracking, as follows.

Algorithm 6: Damped Newton's Method

Initialization: A tolerance parameter $\varepsilon > 0$ and $\mathbf{x}^0 \in \mathbb{R}^n$. (α, β) parameters for the backtracking procedure ($\alpha \in (0, 1), \beta \in (0, 1)$).

General Step: for any $k = 0, 1, 2, \dots$ execute the following steps:

- 1 Compute the Newton direction $\mathbf{d}^k$, which is the solution to the linear system

$$\nabla^2 f(\mathbf{x}^k) \mathbf{d}^k = -\nabla f(\mathbf{x}^k).$$

- 2 Set $t_k = 1$. **while** $\left(f(\mathbf{x}^k) - f(\mathbf{x}^k + t^k \mathbf{d}^k) < -\alpha t^k \nabla f(\mathbf{x}^k)^\top \mathbf{d}^k \right)$ **do**
 - 3 **set** $t^k := \beta t^k$.
 - 4 Set $\mathbf{x}^{k+1} = \mathbf{x}^k + t^k \mathbf{d}^k$.
 - 5 If $\|\nabla f(\mathbf{x}^{k+1})\| \leq \varepsilon$, then STOP and $\mathbf{x}^{k+1}$ is the output.
-

Repeat the last numerical example using damped Newton's method starting from $(10, 10)$.




```

1 function x=pure_newton(f,g,h,x0,epsilon)
2 % Pure Newton's method
3 %
4 % INPUT
5 % =====
6 % f ..... objective function.
7 % g ..... gradient of the objective function
8 % h ..... Hessian of the function
9 % x0..... initial point
10 % epsilon ..... tolerance
11 % OUTPUT
12 % =====
13 % x - solution obtained by Newton's method (up to some tolerance)
14
15 if (nargin<5)
16     epsilon=1e-5;
17 end
18
19 x=x0;
20 gval=g(x);
21 hval=h(x);
22 iter=0;
23 while ((norm(gval)>epsilon)&&(iter<10000))
24     iter=iter+1;
25     x=x-hval\gval;
26     fprintf('iter= %2d f(x)=%10.10f\n',iter,f(x))
27     gval=g(x);
28     hval=h(x);
29 end
30
31 if (iter==10000)
32     fprintf('did not converge')
33 end

```



```

1 function x=newton_backtracking(f,g,h,x0,alpha,beta,epsilon)
2 % Newton's method with backtracking
3 %
4 % INPUT
5 %=====
6 % f ..... objective function
7 % g ..... gradient of the objective function
8 % h ..... hessian of the objective function
9 % x0..... initial point
10 % alpha ..... tolerance parameter for the stepsize selection strategy
11 % beta ..... the proportion in which the stepsize is multiplied
12 % at each backtracking step (0<beta<1)
13 % epsilon ... tolerance parameter for stopping rule
14 % OUTPUT
15 %=====
16 % x ..... optimal solution (up to a tolerance)
17 % of min f(x)
18 % fun_val ... optimal function value
19
20 x=x0;
21 gval=g(x);
22 hval=h(x);
23 d=hval\gval;
24 iter=0;
25 while ((norm(gval)>epsilon)&&(iter<10000))
26     iter=iter+1;
27     t=1;
28     while(f(x-t*d)>f(x)-alpha*t*gval'*d)
29         t=beta*t;
30     end
31     x=x-t*d;
32     fprintf('iter= %2d f(x)=%10.10f\n',iter,f(x))
33     gval=g(x);
34     hval=h(x);
35     d=hval\gval;
36 end
37
38 if (iter==10000)
39     fprintf('did not converge\n')
40 end

```



The Kaczmarz Algorithm

We begin our discussion by studying a classical algorithm proposed by the Polish mathematician Stefan Kaczmarz in 1937, and which was later re-discovered in the 1970s in image processing. This technique was implemented in the very first medical scanners. The Kaczmarz algorithm solves the linear system

$$\mathbf{Ax} = \mathbf{b}$$

by iterating projections along the i -th row of the matrix $\mathbf{A}$, denoted by $\mathbf{a}_i^\top$:

$$\mathbf{x}^{k+1} = \mathbf{x}^k + \frac{b_i - \mathbf{a}_i^\top \mathbf{x}^k}{\|\mathbf{a}_i\|^2} \mathbf{a}_i. \quad (\text{Kaczmarz})$$

Note that this algorithm does not require to compute $\mathbf{A}^{-1}$! In the original Kaczmarz algorithm, the i -th row that is chosen at the k -th iteration of the algorithm is cycled periodically through all the rows of the matrix $\mathbf{A}$, i.e.

$$i = \text{mod}(k, m) + 1,$$

where m is the number of rows of $\mathbf{A}$. Provided that the system is consistent, that is, there exists at least one solution of the system, the iteration $\mathbf{x}^k$ converges to the minimum norm solution of the problem, assuming that $\mathbf{x}^0 = \mathbf{0}$. The convergence analysis of this algorithm remained an open problem until probabilistic methods were introduced by 2009. Nowadays, we can show that the Kaczmarz algorithm converges exponential (and independently of the number of rows) if at the k -th iteration, the i -th row is chosen randomly. This algorithm is known as *Randomized Kaczmarz Algorithm*. We can sample uniformly among the rows, or with a probability that is proportional to the squared row norm $\|\mathbf{a}_i\|^2$, known as *importance sampling*. Figure 7² shows a comparison between cycling (standard Kaczmarz), uniform (simple randomized Kaczmarz), and importance sampling (randomized Kaczmarz) among the rows of a 300 by 100 linear system. The plot shows the evolution of the least squares error, $\|\mathbf{Ax}^k - \mathbf{b}\|$, against the number of iterations (projections) for each algorithm. For every variant of the Kaczmarz algorithm, the method converges to the solution of $\mathbf{Ax} = \mathbf{b}$, however the method converges faster if a randomized row selection is selected instead of a deterministic cycling. For large-scale systems, avoiding the inversion of $\mathbf{A}$ is desirable, so this method can be applied in this case. We recall that solving the linear system $\mathbf{Ax} = \mathbf{b}$ can be cast as the optimization problem

$$\min_{\mathbf{x}} \frac{1}{2m} \|\mathbf{Ax} - \mathbf{b}\|_2^2 = \frac{1}{2m} \sum_{i=1}^m (\mathbf{a}_i^\top \mathbf{x} - b_i)^2,$$

for which a gradient descent method can be constructed as

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \frac{t}{m} \mathbf{A}^\top (\mathbf{Ax} - \mathbf{b}),$$

² Taken from Strohmer, Thomas; Vershynin, Roman (2009), "A randomized Kaczmarz algorithm for linear systems with exponential convergence", *Journal of Fourier Analysis and Applications*, 15 (2): 262–278, arXiv:math/0702226



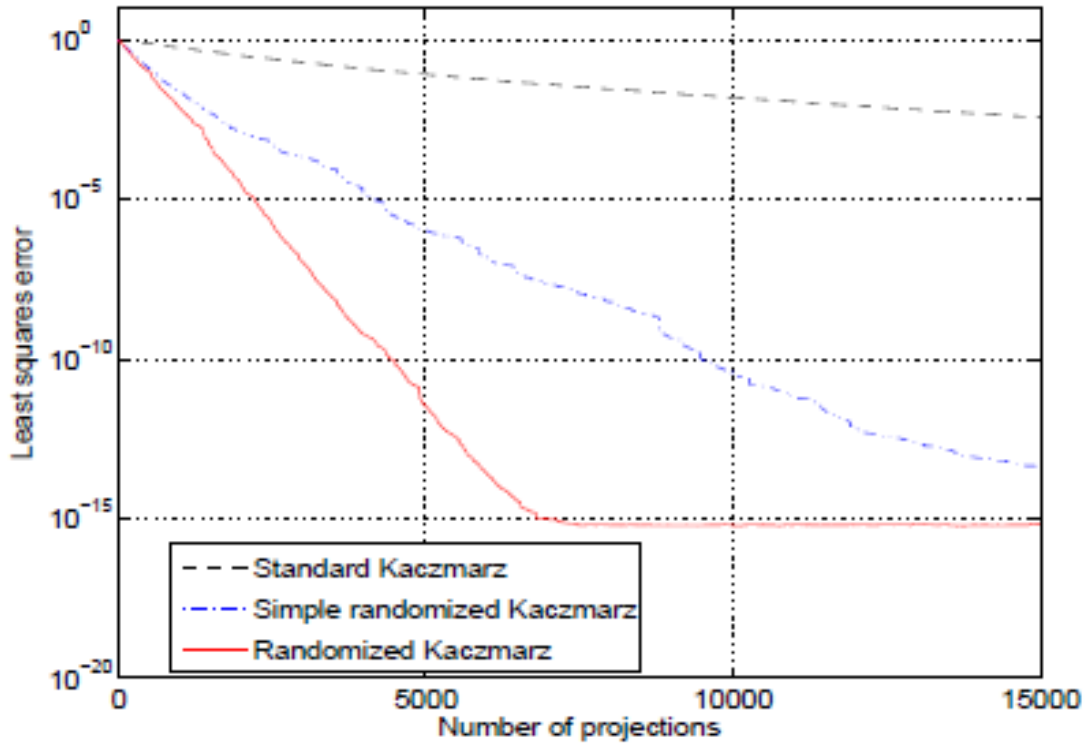


Figure 7: Iteration of the Kaczmarz algorithm for a 300 by 100 linear system of equations, under three different sampling procedures for the rows of $\mathbf{A}$.

and comparing against (Kaczmarz) we can interpret it as a gradient descent where at each iteration, instead of computing the full gradient of

$$\frac{1}{2m} \sum_{i=1}^m (\mathbf{a}_i^\top \mathbf{x} - b_i)^2,$$

we sample a single element of the cost and work with the gradient of

$$\frac{1}{2m} (\mathbf{a}_i^\top \mathbf{x} - b_i)^2.$$

Note that in the case of a uniform sampling among the rows, we can write

$$\frac{1}{2m} \sum_{i=1}^m (\mathbf{a}_i^\top \mathbf{x} - b_i)^2 = \frac{1}{2} \mathbb{E}_i [(\mathbf{a}_i^\top \mathbf{x} - b_i)^2],$$

where the expected value is among a uniform distribution when choosing the vector $[\mathbf{a}_i^\top | b_i]$ uniformly from the rows of the augmented matrix $[\mathbf{A} | \mathbf{b}]$. The generalization of this idea for nonlinear regression problems is what we will discuss in the next section as stochastic gradient descent.

Stochastic Gradient Descent

Stochastic Gradient Descent (SGD) is a fundamental algorithm in machine learning. It is naturally meant for solving nonlinear regression/estimation problems where the cost is of the type

$$\min_{\mathbf{x}} \frac{1}{m} \sum_{i=1}^m Q_i(\mathbf{x}). \quad (3)$$

A natural setting for such an optimization framework arises in nonlinear regression problems.

Nonlinear regression example. Consider the nonlinear model in $\theta \in \mathbb{R}$

$$f(\theta; \mathbf{x}) = x_1 e^{x_2 \theta} \cos(x_3 \theta + x_4),$$

with parameters $\mathbf{x} \in \mathbb{R}^4$ for which we want to find the optimal value $\mathbf{x}^*$ minimizing the norm of the ℓ_2 -error with respect to m observations of the *true* model

$$\hat{f}_i := f(\theta_i), \quad i = 1, \dots, m.$$

We formulate this problem as a nonlinear least squares problem

$$\min_{\mathbf{x}} g(\mathbf{x}) := \frac{1}{m} \sum_{i=1}^m (f(\theta_i; \mathbf{x}) - \hat{f}_i)^2. \quad (\text{NLS})$$

Note that the $\frac{1}{m}$ scaling does not affect the minimizer. Setting

$$Q_i(\mathbf{x}) = (f(\theta_i; \mathbf{x}) - \hat{f}_i)^2$$

we can see how nonlinear regression problems lead to costs or **loss functions** similar to eq. (3). In general, while setting a gradient descent iteration for this problem

$$\mathbf{x}^{k+1} = \mathbf{x}^k - t^k \nabla g(\mathbf{x}^k) = \mathbf{x}^k - \frac{t^k}{m} \sum_{i=1}^m \nabla Q_i(\mathbf{x}^k)$$

is straightforward (following exactly what we discussed in the previous week), when the number of observations m is large³, and the parameter space is high-dimensional ($\mathbf{x} \in \mathbb{R}^n$, $n \gg 1$), the computation of $\sum_{i=1}^m \nabla Q_i(\mathbf{x}^k)$ is overwhelmingly expensive. In the spirit of the Kaczmarz algorithm, stochastic gradient descent circumvents this limitation. Stochastic Gradient Descent dates back to 1951 to the paper by Robbins and Munro “A *stochastic approximation method*”, and instead of computing the gradient with all m points, only a single data point is randomly selected and used. At the next iteration, another randomly selected point is used to compute the gradient and update the solution

$$\mathbf{x}^{k+1} = \mathbf{x}^k - t^k \nabla Q_i(\mathbf{x}^k),$$

³ as in a *big data* framework -think about image datasets, Spotify songs-



where the index i is sampled in each gradient iteration. In machine learning applications, the parameter t^k is known as the **learning rate**. Note that if the samples are drawn uniformly, then

$$g(\mathbf{x}) := \frac{1}{m} \sum_{i=1}^m Q_i(\mathbf{x}) = \mathbb{E}_i[Q_i(\mathbf{x})].$$

and that a random sample constitutes an unbiased estimator of $\nabla g(\mathbf{x})$. The convergence of SGD is stated in the following theorem.

Theorem (Convergence of SGD). *Assume:*

- The cost $g(\mathbf{x})$ is such that

$$\|\nabla g(\mathbf{x}) - \nabla g(\mathbf{y})\| \leq L\|\mathbf{x} - \mathbf{y}\|, \quad \text{and} \quad \nabla^2 g(\mathbf{x}) \geq \mu \mathbf{I}.$$

- The sample gradient $\nabla Q_i(\mathbf{x}^k)$ is an unbiased estimate of $\nabla g(\mathbf{x}^k)$.
- For all $\mathbf{x}$,

$$\mathbb{E}_i[\|Q_i(\mathbf{x})\|^2] \leq \sigma^2 + c\|\nabla g(\mathbf{x})\|^2.$$

Then, if $t^k \equiv t \leq \frac{1}{Lc}$, then SGD achieves

$$\mathbb{E}[g(\mathbf{x}^k) - g(\mathbf{x}^*)] \leq \frac{tL\sigma^2}{2\mu} + (1 - t\mu)^k(g(\mathbf{x}^0) - g(\mathbf{x}^*)).$$

The result above implies:

1. Fast (linear) convergence during the first iterations.
2. Convergence to a neighbourhood of $\mathbf{x}^*$, without further progress.
3. If gradient computation is noiseless, that is $\sigma = 0$, then linear convergence to optimal points.
4. A smaller stepsize t yield better converging points.

The algorithm may require multiple passes through all the data to converge, but each step is now easy to evaluate versus the full computation of the gradient. If instead of a single point, a different subset of points is sampled at each iteration, then we have a **batch gradient descent** algorithm:

$$\mathbf{x}^{k+1} = \mathbf{x}^k - t^k \nabla g(\mathbf{x}^k) = \mathbf{x}^k - \frac{t^k}{|K|} \sum_{i \in K} \nabla Q_i(\mathbf{x}^k),$$

where K denotes a set of p randomly selected datapoints.



Revisiting the Nonlinear Regression Example

If we go back to the model

$$f(\theta; \mathbf{x}) = x_1 e^{x_2 \theta} \cos(x_3 \theta + x_4),$$

during week 5 we explored the Gauss-Newton method for nonlinear regression. We shall modify this setting and implement SGD. Results shown in Figure 8 show the different behaviours observed for different learning rates. Notice the large number of iterations required.

Model:

```
1 function [val]=model(t,x)
2 val= x(1).*exp(x(2).*t).*cos(x(3).*t+x(4)); %%model
```

Gradient of the model with respect to $\mathbf{x}$:

```
1 function [val]=modelgrad(t,x)
2 val(1,1)=exp(x(2).*t).*cos(x(3).*t+x(4));
3 val(2,1)=x(1).*exp(x(2).*t).*t.*cos(x(3).*t+x(4));
4 val(3,1)=-x(1).*exp(x(2).*t).*sin(x(3).*t+x(4)).*t;
5 val(4,1)=-x(1).*exp(x(2).*t).*sin(x(3).*t+x(4));
```

Cost function:

```
1 function [val]=g1(x,tm,fn)
2 m=length(tm);
3 val=norm(model(tm,x)-fn)^2/m;
```

Gradient of the cost function:

```
1 function [val]=gradg1(x,tm,fn)
2 m=length(tm);
3 ind=randi([1 m]); %% sampling among the dataset
4 val=2*(model(tm(ind),x)-fn(ind))*modelgrad(tm(ind),x);
```



Main:

```
1 clear all
2
3 xt=[1;2;pi;0]; %% true parameters
4 tm=[-1:0.001:1]'; %% measurements of the independent variable
5 randn('seed',666);
6 ft=model(tm,xt);%% true model measurements
7 m=length(tm);
8 maxiter=10^4;
9 x0=[1;1;1;1];
10 xm=x0;
11 t=0.01; %% learning rate
12 hist=[g1(xm,tm,fn)];
13 for i=1:maxiter
14 xp=xm-t*gradg1(xm,tm,ft); %%the random sampling is inside gradg1
15 hist=[hist g1(xp,tm,ft)];
16 xm=xp;
17 end
```

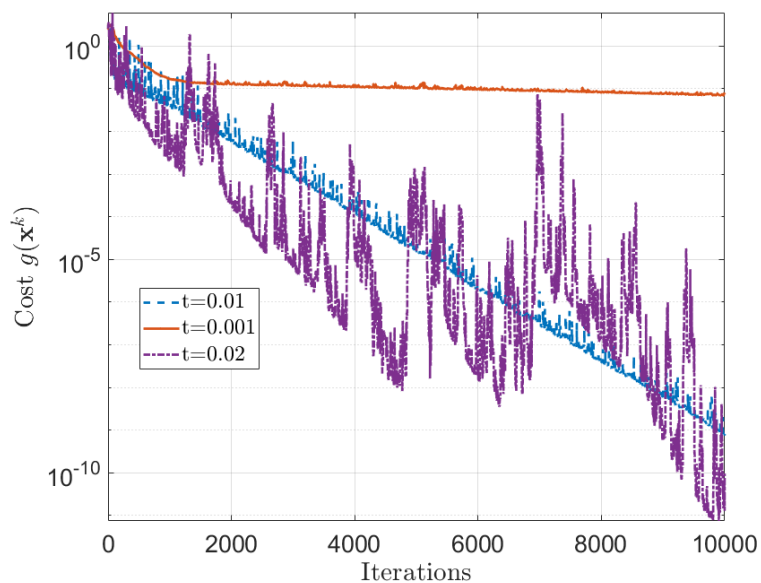


Figure 8: Convergence of the SGD iteration for different learning rates.

